

CS 313 - Project 4

Red Black Tree

Deadline: 11:59PM, Wednesday, December 3, 2025

1 Project Overview

For this project, we will be taking what we've learned in class and using it to implement a Red-Black Tree (RBT). In CS, a red-black tree is a special type of binary tree that self-balances. For this project, I implemented most of the RBT operations and provided them for you via the file **lab4.py**. Your job is to implement the insertion for the RBT. The deletion function can be ignored.

A BST is a Red-Black tree if it satisfies the following Red-Black properties:

- Every node is either red or black.
- The root is always black.
- Red nodes can't have red children (no two red nodes can be adjacent).
- Every path from a node to its descendant NIL nodes has the same number of black nodes.
- The NIL nodes are considered black.

2 Program Requirements:

You will need to complete the RedBlackTree class provided in the file **lab4.py**. To receive any credit, your class must follow the specifications below exactly:

- Class Name: **RedBlackTree**
- Methods to implement:
 - *insert(< RedBlackTree > self, < int > key) → None*
 - * **[50 points]**
 - * Complexity: $O(\log(n))$
 - * Valid Input: an integer indicating a key to add.
 - * Description: This method inserts a new key into the tree. It first inserts the node as in a regular binary search tree, then fixes up the tree so that the red-black properties are maintained.
 - *delete(< RedBlackTree > self, < int > key) → None*
 - * **[50 points]**
 - * Complexity: $O(\log(n))$
 - * Valid Input: an integer indicating a key to remove.
 - * Description: This method removes a node with the given key from the tree. It first removes the node as in a regular binary search tree, then fixes up the tree so that the red-black properties are maintained.

3 Submission Requirements:

Submit a single file **lab4.py** to canvas.